

Code Explanation

Code Explanation: Customer Order Pipeline ETL Job Unit Tests
This test suite validates an AWS Glue PySpark ETL pipeline that processes customer and order data. The tests verify data reading, quality transformations, business logic, and data writing operations.---

Test Setup and Initialization

Spark Session Creation

```
def get_spark():
```

- Creates or retrieves a local Spark session for testing- Configured with 2 local executors and 2 shuffle partitions for performance- Sets log level to ERROR to reduce test output noise- Checks if Spark already exists in builtins to avoid duplicate sessions---

Test Categories

1. Configuration Management Tests

test_config_manager_loads_parameters() **Purpose**: Validates that job parameters load correctly from configuration- **Verifies**: - Parameters dictionary is not None - Contains required S3 path keys for customer and order data - Paths match expected S3 locations (`s3://adif-sdlc/sdlc_wizard/`)---

2. Data Reading Tests

test_s3_data_reader_reads_customer_data() **Purpose**: Tests customer data reading and schema normalization- **Process**: 1. Creates mock customer data with 3 records 2. Mocks internal CSV reading method to return test data 3. Calls `read_data_safe()` with expected schema- **Verifies**: - Returns non-null DataFrame - Contains exactly 3 records - Column names are normalized to lowercase - Logger records successful load
test_s3_data_reader_reads_order_data() **Purpose**: Tests order data reading with numeric and date fields- **Process**: Similar to customer test but with order schema- **Verifies**: - Handles mixed data types (strings, doubles, integers, dates) - Returns 3 records with correct lowercase column names
test_s3_data_reader_validates_schema() **Purpose**: Tests schema validation failure handling- **Process**: 1. Creates data missing required `EmailId` column 2. Attempts to read with full expected schema- **Verifies**: - Raises `ValueError` with "Schema validation failed" message - Logs error appropriately---

3. Data Quality Tests

test_data_quality_transformer_removes_nulls() **Purpose**: Tests NULL value removal logic- **Test Data**: - 4 records: 1 clean, 1 with NULL name, 1 with "Null" string, 1 with NULL custid- **Verifies**: - Only 1 record remains (the clean one) - Removes both actual NULLs and "Null" strings - Logs transformation
test_data_quality_transformer_removes_duplicates() **Purpose**: Tests duplicate record removal- **Test Data**: - 4 records with 1 exact duplicate- **Verifies**: - Returns 3 unique records - Preserves first occurrence of duplicates---

4. Data Writing Tests

test_s3_data_writer_writes_parquet() **Purpose**: Tests Parquet format writing- **Process**: 1. Creates sample customer DataFrame 2. Mocks internal Parquet write method 3. Calls `write_data_safe()` with table and database names- **Verifies**: - Write method is invoked - Logger records write operation

test_s3_data_writer_writes_hudi_scd2() **Purpose**: Tests Hudi format writing with Slowly Changing Dimension Type 2 (SCD2) support- **Process**: 1. Creates order summary DataFrame 2. Provides Hudi configuration (table name, record key, precombine field, partition, operation) 3. Mocks internal Hudi write method- **Verifies**: - Write method is called - SCD2 columns are added: `IsActive`, `StartDate`, `EndDate`, `OpTs` - These columns enable tracking historical changes---

5. Business Logic Tests

test_order_summary_processor_creates_summary() **Purpose**: Tests order summary creation by joining customer and order data- **Process**: 1. Creates 2 customers and 2 orders 2. Calls `create_order_summary()` to join datasets- **Verifies**: - Returns 2 joined records - Contains `TotalAmount` calculated column - Includes customer details (name, emailid, region) - **Calculation validation**: - Order O001: $1200.00 \times 1 = 1200.00$ - Order O002: $25.50 \times 2 = 51.00$

test_aggregation_processor_calculates_spend() **Purpose**: Tests customer spending aggregation by date- **Test Data**: - Customer C001: 3 orders across 2 dates - Customer C002: 1 order on 1 date- **Verifies**: - Returns 3 aggregated records (C001 on 2 dates + C002 on 1 date) - Contains `TotalSpend` column - **Aggregation validation**: - C001 on 2024-01-15: $1200.00 + 51.00 = 1251.00$ - C001 on 2024-01-16: 75.00 - C002 on 2024-01-15: 300.00---

6. End-to-End Integration Test

test_end_to_end_pipeline() **Purpose**: Tests complete pipeline flow from ingestion to analytics- **Pipeline Steps**: 1. **Data Ingestion**: - Reads customer data (4 records with 1 null, 1 duplicate) - Reads order data (4 records with 1 null, 1 duplicate) 2. **Data Quality**: - Removes NULL values from both datasets - Removes duplicate records - **Result**: 2 clean customers, 2 clean orders 3. **Business Transformation**: - Creates order summary by joining customers and orders - **Result**: 2 enriched order records 4. **Analytics Aggregation**: - Calculates total spend per customer per date - **Result**: 2 aggregated records 5. **Data Writing**: - Writes cleaned customer data to Parquet - Writes cleaned order data to Parquet - Writes order summary to Hudi with SCD2 columns - Writes analytics aggregation to Parquet- **Verifies**: All components work together correctly with realistic data quality issues---

Test Execution

The `if __name__ == "__main__":` block runs all tests sequentially and prints results:- Each test prints a checkmark (✓) on success- Final message confirms all tests passed- Provides simple test runner without external frameworks---

Key Testing Patterns

1. **Mocking**: Uses `unittest.mock` to isolate components and avoid actual S3 I/O. 2. **Sample Data**: Creates realistic test data with edge cases (nulls, duplicates) 3. **Assertion Patterns**: Validates counts, column presence, calculated values, and error handling 4. **Integration Testing**: End-to-end test validates complete workflow 5. **Local Spark**: Uses local Spark session for fast, isolated testing